

AI-Powered Network Attack Detection System and Analyst Dashboard

Göktuğ VARAN 

Department of Computer Engineering
Biruni University, Istanbul, Turkey
210408029@st.biruni.edu.tr

Mehmet Taha YILMAZ 

Department of Computer Engineering
Biruni University, Istanbul, Turkey
210408039@st.biruni.edu.tr

Prof. Dr. Özgür Koray ŞAHİNGÖZ 

Department of Computer Engineering
Biruni University, Istanbul, Turkey
osahingoz@biruni.edu.tr

Abstract—This paper presents an AI-powered network intrusion detection prototype that combines machine learning-based traffic classification with a backend API and an analyst-oriented dashboard. The system is developed using the CIC-IDS2017 dataset and supports both binary classification, where network traffic is classified as benign or attack, and multiclass classification, where the specific attack category is predicted. The proposed workflow includes data preprocessing, feature engineering, leakage analysis, model training, cross-validation, performance evaluation, model comparison, explainability support, a FedAvg-based federated learning simulation, and prototype integration. Random Forest, XGBoost, and CatBoost models are evaluated using accuracy, weighted F1-score, macro F1-score, confusion matrices, and per-class F1-score. Experimental results show that the models achieve strong performance under per-file evaluation, while time-based evaluation is more challenging due to temporal distribution shift. A FedAvg-based federated learning simulation across five clients achieves an accuracy of 0.9695 and a weighted F1-score of 0.9672 using a three-layer MLP without sharing raw training data. The final prototype integrates the selected Random Forest model into a FastAPI backend and visualizes prediction results through a dashboard with traffic flow, attack distribution, recent alerts, and SHAP-based explainability information.

Index Terms—Intrusion detection system, network security, machine learning, CIC-IDS2017, Random Forest, XGBoost, CatBoost, federated learning, FedAvg, SHAP, analyst dashboard.

I. INTRODUCTION

Network security has become an important requirement for modern information systems because organizations depend heavily on computer networks, cloud services, web applications, and connected devices. As the number and complexity of cyberattacks increase, traditional security mechanisms may not be sufficient to detect malicious behavior in a timely and understandable way. Network Intrusion Detection Systems (NIDS) are therefore used to monitor network traffic and identify suspicious or malicious activities.

Machine learning has become a widely used approach for intrusion detection because it can learn patterns from historical network traffic and classify new traffic records automatically. Recent studies show that machine learning and deep learning methods can improve detection performance for both traditional and emerging attack types [1], [2], [3]. However, intrusion detection datasets often contain class imbalance, noisy labels, dataset-specific artifacts, and temporal distribution shifts. These issues can produce overly optimistic results if the evaluation strategy is not carefully designed [4], [5], [6].

Although many IDS studies report high offline classification performance, several limitations remain in practical use. First, benchmark-based evaluation may produce optimistic results when dataset artifacts or leakage-sensitive features are not carefully examined. Second, many studies focus only on model accuracy and do not provide an integrated software prototype. Third, analysts need interpretable and usable outputs rather than only prediction labels or metric tables. Fourth, privacy-preserving distributed training is increasingly relevant in real-world IDS deployments where raw traffic data cannot be centralized. These limitations motivate the development of an end-to-end IDS prototype that combines model evaluation, leakage-aware feature analysis, backend deployment, dashboard visualization, explanation support, and federated learning experimentation.

This project develops an AI-powered real-time-like network attack detection system and analyst dashboard. The system is not limited to offline model training; it also integrates the trained model into a working software prototype. The backend API serves prediction, history, statistics, and explainability outputs, while the frontend dashboard visualizes attack status, traffic flow, attack distribution, recent events, and feature-based explanations. In addition, a FedAvg-based federated learning simulation demonstrates distributed model training without sharing raw client data.

The main contributions of this work are as follows:

- Development of a machine learning-based intrusion detection workflow for binary and multiclass classification using 45 selected flow-based features from CIC-IDS2017.
- Comparison of Random Forest, XGBoost, and CatBoost models under per-file and time-based evaluation scenarios to highlight the impact of temporal distribution shift.
- Leakage-oriented feature analysis to reduce the risk of overly optimistic evaluation results caused by dataset-specific artifacts.
- Cross-validation using both stratified and standard 5-fold strategies to verify model stability across different data partitions.
- A FedAvg-based federated learning simulation across five distributed clients using a three-layer MLP without sharing raw training data.
- Integration of the selected model into a FastAPI backend and analyst dashboard with SHAP-based explainability

support.

II. RELATED WORK

Intrusion detection has been studied extensively using both traditional machine learning and deep learning methods. Gümüşbaş *et al.* [1] presented a comprehensive survey of cybersecurity datasets and deep learning methods for IDS research. Lansky *et al.* [2] reviewed deep learning-based intrusion detection systems and discussed common architectures such as autoencoders, convolutional neural networks, recurrent neural networks, and hybrid models. Anis *et al.* [3] further surveyed recent deep learning applications in NIDS, including transformer-based approaches and robustness issues.

Several studies have focused on CIC-IDS2017 because it is one of the most widely used benchmark datasets for network intrusion detection. Maseer *et al.* [7] benchmarked machine learning algorithms for anomaly-based IDS on CIC-IDS2017. Singh and Srivastav [8] proposed a machine learning framework for anomaly detection using the same dataset. Engelen *et al.* [4] identified important dataset quality issues in CIC-IDS2017, and Liu *et al.* [5] examined error prevalence in CIC-IDS2017 and CSE-CIC-IDS2018. These studies show that dataset artifacts and labeling errors should be considered when interpreting IDS results. Cantone *et al.* [6] further demonstrated that IDS models may face significant performance degradation under cross-dataset and temporal generalization scenarios.

Recent work also investigates real-time, explainable, federated, and zero-day intrusion detection. El-Hajj [9] studied AI-powered real-time threat detection using network traffic data. Banerjee *et al.* [10] proposed a hybrid ML-DL approach for real-time anomaly detection. Roshan and Zafar [11] used Kernel SHAP to optimize and explain network anomaly detection models. Mothukuri *et al.* [12] demonstrated federated learning for anomaly detection in IoT security attacks, while Aouedi *et al.* [13] proposed FLUIDS, a federated semi-supervised IDS. Belarbi *et al.* [14] investigated federated deep learning for IoT network intrusion detection. Wu *et al.* [15] presented RTIDS, a robust transformer-based IDS using self-attention. Other studies have addressed transfer learning at the edge [16], imbalanced intrusion detection [17], and zero-day attack detection [18].

Compared with existing studies, the proposed system contributes in three ways that are rarely combined in a single prototype: leakage-aware multi-scenario evaluation, end-to-end software integration with SHAP explainability, and a FedAvg-based federated learning extension evaluated on the same dataset.

III. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

The methodology of the proposed system consists of dataset preparation, data cleaning, feature engineering, leakage analysis, model training, cross-validation, model evaluation, model export, backend API development, dashboard implementation,

federated learning simulation, and system testing. The general workflow is shown in Fig. 1.



Figure 1: General Methodology Workflow of the Proposed IDS Prototype

A. System Architecture and Data Flow

The proposed system consists of three main components: the machine learning pipeline, the backend API, and the analyst dashboard. The machine learning pipeline is responsible for data preprocessing, feature engineering, leakage analysis, model training, evaluation, and model export. The backend API loads the exported model files and provides prediction, recent event history, traffic statistics, and explainability outputs. The analyst dashboard consumes these backend outputs and presents them in an interface designed for monitoring and investigation.

The overall architecture follows a modular structure. The model layer produces binary and multiclass predictions, the backend layer transforms these outputs into structured API responses, and the frontend layer visualizes the results for the analyst. This separation makes the prototype easier to extend with live traffic ingestion, database storage, authentication, notification mechanisms, or SIEM integration in future work.

B. Dataset Preparation and Preprocessing

The CIC-IDS2017 dataset was used as the main data source. It contains benign traffic and multiple attack categories captured in a controlled network environment. Each record represents a network flow and includes statistical features extracted from packet-level communication, such as flow duration, packet length statistics, byte counts, packet counts, and inter-arrival time measurements.

The preprocessing stage included cleaning column names, removing irrelevant or duplicated fields, handling missing and infinite values, encoding labels, and preparing separate targets for binary and multiclass classification. For binary classification, labels were converted into two classes: benign and attack. For multiclass classification, the original attack

categories were preserved so that the model could learn class-specific attack patterns.

A controlled sampling strategy was applied to reduce class dominance without synthetic oversampling. At most 50,000 benign records were selected per daily traffic file, and at most 30,000 records were selected per attack class where available. This approach preserved real traffic records while improving class balance.

Because intrusion detection datasets are often imbalanced, the evaluation was not based only on accuracy. Weighted F1-score, macro F1-score, confusion matrices, and per-class F1-score values were also used to obtain a more reliable understanding of model behavior, especially for minority attack classes.

C. Feature Engineering and Leakage Control

Feature engineering was performed to improve the reliability and efficiency of the model training process. Flow-based network datasets may include redundant, highly correlated, sparse, skewed, or unstable features. If such features are used without analysis, the model may learn misleading patterns or become overly dependent on dataset-specific characteristics.

The feature engineering pipeline applied the following steps: (1) removal of duplicate columns, (2) high-correlation filtering using a threshold of 0.98, (3) mutual information analysis to identify features with low predictive value (threshold 0.01), and (4) skewness analysis to detect unstable or artifact-prone features. Starting from approximately 78 raw flow-based features, this pipeline resulted in a final set of 45 features used for model training and export.

Leakage-oriented feature analysis was also performed to reduce the risk of overly optimistic evaluation. In CIC-IDS2017, certain features may reflect dataset-specific generation patterns rather than general attack behavior. Address-based identifiers, flow IDs, and timestamp-related fields were checked, and `Destination Port` was removed from the final feature set because it may encode dataset-specific service configurations rather than behavioral attack characteristics. Additionally, `min_seg_size_forward` and `Bwd Header Length` were excluded due to extreme skewness values (approximately -681 and -578 respectively), which are consistent with known CICFlowMeter and Wireshark extraction artifacts [4], [5].

All feature selection steps were performed on training data only using a temporary 80/20 split, preventing any leakage from test set statistics into the feature selection process.

D. Model Training and Evaluation Strategy

Three tree-based ensemble models were evaluated: Random Forest, XGBoost, and CatBoost. These models were selected because they perform well on tabular network-flow datasets and can model non-linear relationships between traffic features. The models were trained separately for binary and multiclass classification tasks.

The binary classification task evaluates whether a traffic record is benign or malicious. The multiclass classification task

evaluates whether the model can identify the specific attack category. The models were compared using accuracy, weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and per-class F1-score analysis.

Two primary evaluation scenarios were considered. The *per-file split* uses a stratified 80/20 division of the dataset, keeping similar class distributions in training and testing. The *time-based split* separates training and testing data according to the chronological order of the CICIDS2017 capture days (Monday through Wednesday for training, Thursday and Friday for testing), providing a more realistic scenario that tests temporal generalization.

In addition, both stratified and standard 5-fold cross-validation were applied to the Random Forest model to verify stability across different data partitions. Stratified cross-validation preserves class proportions in each fold and is particularly important for imbalanced multiclass datasets. Comparing stratified and standard results confirmed that observed performance was consistent and not strongly dependent on a single train-test split.

E. Backend Integration and Dashboard Implementation

After model evaluation, the selected Random Forest models were exported as model bundles containing the trained classifier and the corresponding 45-feature list. The backend was implemented using FastAPI. It loads the binary and multiclass models at startup, initializes SHAP TreeExplainer objects for explainability, and serves prediction, history, statistics, and explanation data through dedicated API endpoints.

When the stream endpoint is called, the backend samples a preprocessed traffic record, generates binary and multiclass predictions, and when an attack is detected, computes the top-5 most influential features using per-sample SHAP values. The explanation output includes the feature name, SHAP value, actual feature value, contribution direction, and a security-oriented description. Recent events are stored in an in-memory queue to support dashboard history and statistics without requiring a persistent database.

The analyst dashboard was implemented using React and Next.js. It displays the current prediction status, real-time-like traffic flow, attack distribution, recent security events, and SHAP-based explanation outputs. The explainability component helps analysts understand which features contributed to a prediction instead of only showing the final class label.

The integrated prototype was tested in a local development environment. Tests verified that the exported models were loaded correctly, prediction endpoints returned structured outputs, and the dashboard correctly displayed prediction results, traffic statistics, recent alerts, and explainability information.

F. Federated Learning Simulation

As an extension of the centralized IDS pipeline, a FedAvg-based federated learning simulation was implemented to investigate distributed model training without sharing raw client data. This extension is motivated by privacy-preserving IDS scenarios such as multi-organization or IoT environments

Table I: Binary Classification Performance Comparison

Model	Split	Acc.	W-F1	Macro F1
Random Forest	Per-file	0.9985	0.9986	0.9980
Random Forest	Time-based	0.8540	0.8197	0.6800
XGBoost	Per-file	0.9982	0.9982	0.9975
CatBoost	Per-file	0.9982	0.9982	0.9976

Table II: Multiclass Classification Performance Comparison

Model	Split	Acc.	W-F1	Macro F1
Random Forest	Per-file	0.9969	0.9969	0.9187
Random Forest	Time-based	0.7957	0.7119	0.0745
XGBoost	Per-file	0.9973	0.9970	0.9025
CatBoost	Per-file	0.9629	0.9710	0.8152

where centralizing raw network traffic is impractical [12], [13], [14].

The federated learning model uses a three-layer Multi-Layer Perceptron (MLP) with hidden dimensions of 128 and 64, ReLU activations, and dropout regularization. The dataset was divided into a global holdout test set (20%, stratified) and a training partition distributed across five clients using IID splitting, which preserves approximate class proportions across clients. Each communication round consists of local training for two epochs per client, followed by sample-weighted FedAvg aggregation on the central server.

Training was performed for five communication rounds using the same 45 selected features as the centralized models. The global model was evaluated on the holdout test set after each round. This setup follows the standard FedAvg protocol [12], [13]: global model weights are broadcast to clients, local updates are computed independently, and aggregated weights are returned to the server without exchanging raw training data.

It should be noted that the federated simulation uses IID client partitioning and a homogeneous MLP architecture across all clients, which represents an idealized federated setting. Non-IID distributions, heterogeneous client capabilities, and communication cost analysis are identified as directions for future work.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the experimental results obtained from the machine learning models, the federated learning simulation, and the developed intrusion detection prototype.

A. Overall Model Performance

Table I presents the binary classification performance. Table II presents the multiclass classification performance.

As shown in Table I, all per-file binary models achieved high performance. Random Forest obtained an accuracy of 0.9985 and a weighted F1-score of 0.9986. The time-based Random Forest result was notably lower (accuracy 0.8540, macro F1 0.6800), confirming that temporal distribution shift makes the detection task significantly more difficult.

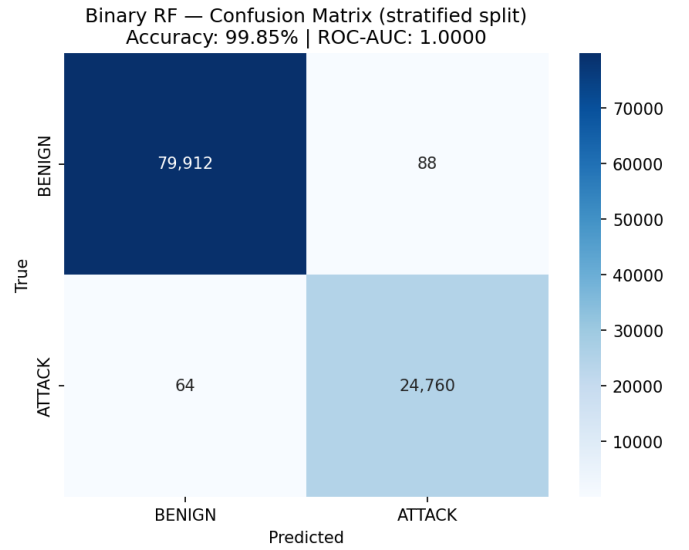


Figure 2: Binary Classification Confusion Matrix (Per-File, Stratified Split)

As shown in Table II, XGBoost achieved the highest per-file multiclass accuracy (0.9973), while Random Forest achieved the highest macro F1-score (0.9187). Since macro F1-score gives equal importance to all classes and better reflects behavior on minority attack categories, Random Forest was selected as the final model for prototype integration. The time-based multiclass macro F1-score dropped sharply to 0.0745, which reflects both the challenge of temporal generalization and the effect of temporal class distribution shifts in CIC-IDS2017, where some attack categories are concentrated on specific capture days and may be absent or severely underrepresented in the training split.

B. Cross-Validation Results

To verify model stability, both stratified and standard 5-fold cross-validation were applied to the Random Forest model for binary and multiclass tasks. Stratified cross-validation consistently produced results within 0.001 of the per-file split results, confirming that the dataset is large enough for random splits to naturally preserve class proportions. The negligible variance across folds and evaluation strategies indicates that the reported performance is not dependent on a specific data partition.

C. Binary Classification Results

Fig. 2 shows the confusion matrix of the Random Forest binary classifier under the per-file evaluation setting. The model correctly classified 79,912 benign and 24,760 attack samples, with only 88 false positives and 64 false negatives, yielding a false positive rate of approximately 0.11% and a false negative rate of approximately 0.26%.

D. Multiclass Classification Results

Fig. 3 presents the multiclass confusion matrix of the Random Forest classifier. Most major attack classes including

Table III: FedAvg Federated Learning Results per Communication Round

Round	Acc.	W-F1	Macro F1
1	0.9350	0.9232	0.5105
2	0.9594	0.9556	0.6890
3	0.9635	0.9597	0.6938
4	0.9683	0.9651	0.7232
5	0.9695	0.9672	0.7451

DDoS, DoS Hulk, DoS GoldenEye, FTP-Patator, SSH-Patator, and PortScan were classified correctly. However, confusion was observed for minority web attack classes, consistent with their lower per-class F1-scores.

E. Per-Class F1-Score Analysis

Fig. 4 shows the per-class F1-score comparison across Random Forest, XGBoost, and CatBoost. Most major attack classes achieved F1-scores above 0.90 across all models. Web Attack-XSS and Web Attack-Brute Force achieved notably lower scores due to limited training samples and overlapping flow-based feature patterns with other attack types. This confirms that macro F1-score and per-class analysis are necessary for a reliable understanding of multiclass IDS model behavior, as overall accuracy alone can be misleading under class imbalance.

F. Federated Learning Results

Table III summarizes the results of the FedAvg-based federated learning simulation. The global MLP model was evaluated on the holdout test set (104,824 samples) after each of the five communication rounds.

After five communication rounds, the global federated model achieved an accuracy of 0.9695, a weighted F1-score of 0.9672, and a macro F1-score of 0.7451. The progressive improvement across rounds demonstrates stable FedAvg convergence under IID client partitioning. Although the federated macro F1-score is lower than the centralized Random Forest result (0.9187), the federated model achieves this performance without any client sharing raw training data, demonstrating the feasibility of privacy-preserving distributed intrusion detection.

The lower macro F1-score in the federated setting compared to centralized Random Forest is expected for two reasons. First, the federated model uses a neural network architecture that requires more training data per class to learn stable decision boundaries, which disadvantages minority classes. Second, the IID client partitioning in this experiment represents an idealized scenario; non-IID distributions would likely further reduce minority class performance.

G. Dashboard Output

The developed dashboard visualizes prediction results generated by the backend API, including the current prediction status, traffic flow over time, attack type distribution, recent security events, and SHAP-based feature explanations for

detected attacks. The dashboard demonstrates that the trained models can be served outside the notebook environment and converted into analyst-oriented visual information supporting real-time-like security monitoring.

H. Discussion and Limitations

The experimental results show that the proposed system performs strongly under per-file evaluation. All three models achieved accuracy above 0.99 and macro F1-score above 0.80 for binary classification. For multiclass classification, the per-file macro F1-scores ranged from 0.80 to 0.92 across models, reflecting good but not uniform performance on minority classes.

The time-based evaluation results are substantially lower, particularly for multiclass classification. This is an expected finding because CIC-IDS2017 has a chronological structure where specific attack types appear only on certain capture days. When earlier days are used for training and later days for testing, the model encounters attack classes that were rarely seen during training, causing the macro F1-score to drop sharply. This finding supports the argument that random or per-file evaluation strategies may produce overly optimistic intrusion detection results when temporal characteristics are ignored.

Random Forest was selected for prototype integration over XGBoost because it produced the highest macro F1-score and offers simpler deployment with direct compatibility with SHAP TreeExplainer. Although XGBoost produced marginally higher accuracy in the multiclass per-file setting, the difference was within 0.0004 in accuracy and Random Forest led by 0.0162 in macro F1, which is the more informative metric for minority class behavior.

The leakage analysis confirmed that the selected feature set did not contain explicit address-based identifiers or flow ID fields. The removal of Destination Port, min_seg_size_forward, and Bwd Header Length reduced the risk of dataset-specific shortcut learning and contributed to a more reliable evaluation.

The current prototype has several limitations. Real-time monitoring is simulated using preprocessed holdout records rather than live packet capture. Event history is stored in memory without persistent database support. The federated simulation uses IID partitioning, which does not fully represent realistic federated scenarios with heterogeneous data distributions. Additionally, the federated MLP model is not yet integrated into the production backend, which currently uses the centralized Random Forest models exclusively.

V. CONCLUSION AND FUTURE WORK

This paper presented an AI-powered network attack detection prototype with an analyst-oriented dashboard. The system combined a machine learning pipeline, FastAPI backend, and React/Next.js dashboard to support binary and multiclass intrusion detection on the CIC-IDS2017 dataset. The workflow included leakage-oriented feature analysis, multi-scenario

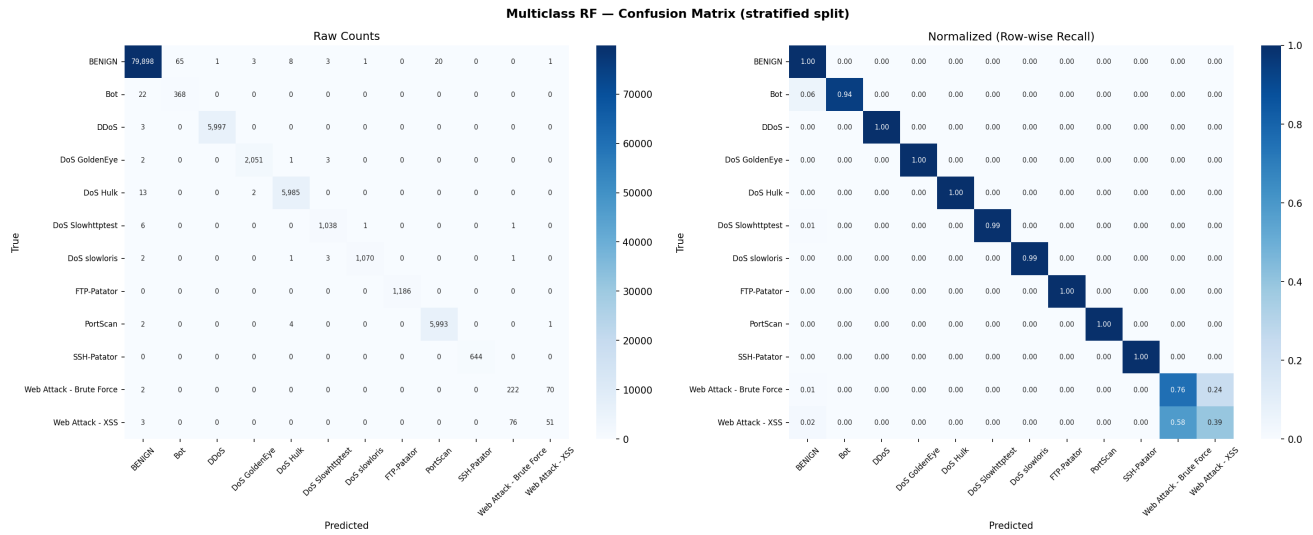


Figure 3: Multiclass Classification Confusion Matrix (Per-File, Stratified Split)

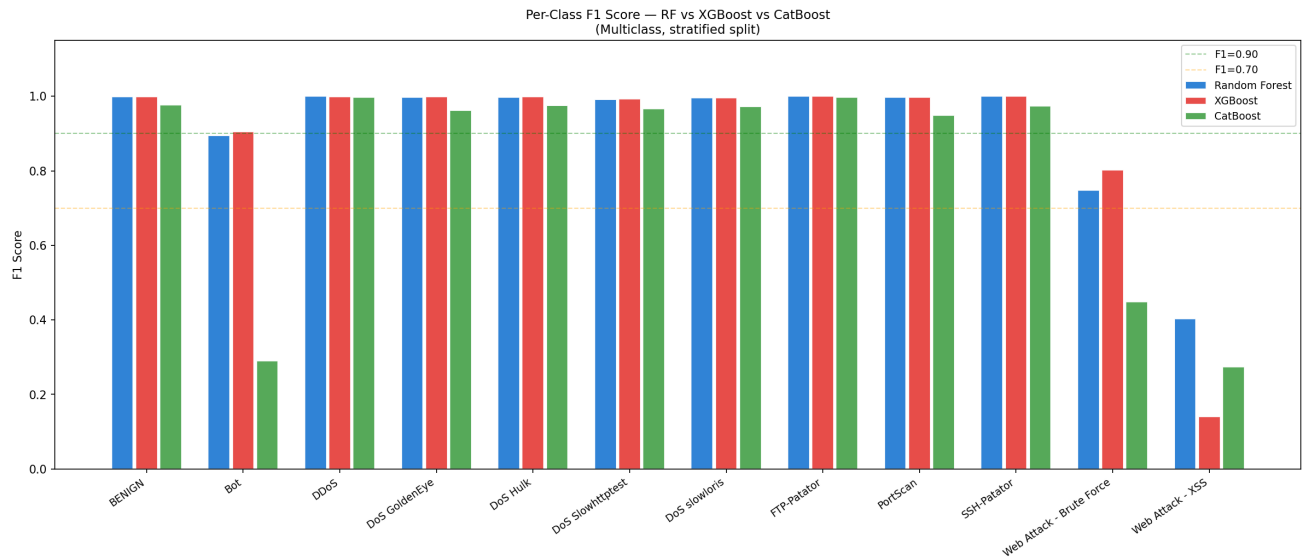


Figure 4: Per-Class F1-Score Comparison (Multiclass, Stratified Split)

evaluation with per-file and time-based splits, 5-fold cross-validation, a FedAvg-based federated learning simulation, and SHAP-based explainability integration.

The experimental results showed that Random Forest, XGBoost, and CatBoost achieved strong performance under per-file evaluation. The time-based evaluation demonstrated that temporal distribution shift remains an important challenge for realistic IDS evaluation. Random Forest was selected for prototype integration because it achieved the highest macro F1-score (0.9187 for multiclass per-file) and provided a practical balance between performance, stability, deployment simplicity, and explainability compatibility. The FedAvg-based federated learning simulation achieved an accuracy of 0.9695 and a macro F1-score of 0.7451 after five communication rounds, demonstrating that effective intrusion detection is feasible

without centralizing raw training data.

The prototype demonstrates that offline IDS models can be transformed into a working monitoring system by combining backend prediction services, dashboard visualization, and SHAP-based explanation outputs. The system provides useful support for analysts by presenting prediction results, attack categories, recent alerts, and feature-based explanations in a single interface.

Future work will focus on four directions: (1) integrating live packet capture instead of preprocessed traffic records using tools such as CICFlowMeter or Zeek; (2) adding persistent database storage and SIEM integration for operational use; (3) improving detection performance for minority web attack classes through additional data balancing, feature enrichment, and specialized model retraining; and (4) extending the fed-

erated learning simulation with non-IID client distributions, larger client populations, communication-cost analysis, and deployment of the federated model into the production backend.

REFERENCES

- [1] D. Gümüşbaş, T. Yıldırım, A. Genovese, and F. Scotti, "A comprehensive survey of databases and deep learning methods for cybersecurity and intrusion detection systems," *IEEE Systems Journal*, vol. 15, no. 2, pp. 1717–1731, 2021.
- [2] J. Lansky, S. Ali, M. Mohammadi, M. K. Majeed, S. H. T. Karim, S. Rashidi, M. Hosseinzadeh, and A. M. Rahmani, "Deep learning-based intrusion detection systems: A systematic review," *IEEE Access*, vol. 9, pp. 101 574–101 599, 2021.
- [3] F. M. Anis, M. Alabdullatif, S. Aljbli, and M. Hammoudeh, "A survey on the applications of deep learning in network intrusion detection systems to enhance network security," *IEEE Access*, vol. 13, pp. 185 357–185 373, 2025.
- [4] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the cicids2017 case study," in *2021 IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.
- [5] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018," in *2022 IEEE Conference on Communications and Network Security (CNS)*, 2022, pp. 254–262.
- [6] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144 489–144 508, 2024.
- [7] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the cicids2017 dataset," *IEEE Access*, vol. 9, pp. 22 351–22 370, 2021.
- [8] R. Singh and G. Srivastav, "Novel framework for anomaly detection using machine learning technique on cic-ids2017 dataset," in *2021 International Conference on Technological Advancements and Innovations (ICTAI)*, 2021, pp. 632–636.
- [9] M. El-Hajj, "Ai-powered threat detection and response: Leveraging machine learning for real-time intrusion detection systems (ids) using network traffic data," in *2025 5th Intelligent Cybersecurity Conference (ICSC)*, 2025, pp. 84–90.
- [10] S. Banerjee, B. Basak, S. Mandal, S. Manna, S. Chakraborty, S. Ghatak, and A. Das, "Real-time anomaly detection in network traffic: A hybrid ml-dl approach," in *2025 International Conference on Engineering Innovations and Technologies (ICoEIT)*, 2025, pp. 536–539.
- [11] K. Roshan and A. Zafar, "Using kernel shap xai method to optimize the network anomaly detection model," in *2022 9th International Conference on Computing for Sustainable Global Development (INDIACom)*, 2022, pp. 74–80.
- [12] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for iot security attacks," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2545–2554, 2022.
- [13] O. Aouedi, K. Piamrat, G. Muller, and K. Singh, "Fluids: Federated learning with semi-supervised approach for intrusion detection system," in *2022 IEEE 19th Annual Consumer Communications Networking Conference (CCNC)*, 2022, pp. 523–524.
- [14] O. Belarbi, T. Spyridopoulos, E. Anthi, I. Mavromatis, P. Carnelli, and A. Khan, "Federated deep learning for intrusion detection in iot networks," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 237–242.
- [15] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "Rtids: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64 375–64 387, 2022.
- [16] Z. Ali, W. Tiberti, A. Marotta, and D. Cassioli, "A lightweight intrusion detection system for iot based on deep transfer learning at the edge," in *2025 IEEE 9th Forum on Research and Technologies for Society and Industry (RTSI)*, 2025, pp. 1–6.
- [17] M. Mbow, H. Koide, and K. Sakurai, "An intrusion detection system for imbalanced dataset based on deep learning," in *2021 Ninth International Symposium on Computing and Networking (CANDAR)*, 2021, pp. 38–47.
- [18] N. Mearaj and M. A. Wani, "Zero-day attack detection with machine learning and deep learning," in *2023 10th International Conference on Computing for Sustainable Global Development (INDIACom)*, 2023, pp. 719–725.